



REST API: Vejrdata

Opgavebeskrivelse

Specialisterne har i en tidligere opgave fået opbygget en ETL-pipeline, der løbende indsamler og gemmer vejr- og miljødata i et PostgreSQL data warehouse. Data kommer fra både DMIs officielle målestationer, og Specialisternes egne instrumenter i Ballerup, og kombinationen giver et unikt billede af både udendørs vejrforhold og indendørs arbejdsmiljø.

Nu ønsker Specialisterne at gøre dataene tilgængelige for resten af organisationen. Facilities-teamet vil følge indendørs luftkvalitet, og ledelsen ønsker at integrere vejrdata i en kommende dashboard-løsning. Frem for at give direkte adgang til databasen skal der bygges et REST API, der udstiller de relevante data på en kontrolleret og veldokumenteret måde. Jeres opgave er at designe og implementere netop det. Dataindsamlingen er allerede løst – I skal bygge broen mellem data warehouse og de teams, der har brug for dataene.

Afleveringsformat

- Github
 - Kildekode i et Git-repository.
 - README med opstartsvejledning
 - OpenAPI-dokumentation
 - Præsentation
 - Forbered en 10 minutters præsentation af dit REST API. Nedenfor er nogle forslag til fokusområder:
 - Projektstyring
 - Centrale kodeelementer og teknologivalg (inkluder gerne snippets)
 - Udfordringer og læringer
 - Demo
-

Forslag til fremgangsmåde

API'et skal gøre det nemt for en klient – fx en frontend-applikation eller et analyseredskab – at hente og filtrere miljødata.

Sprog og framework: I vælger selv sprog og framework, men det skal begrundes. Oplagte valg er f.eks. Python med FastAPI eller Flask, eller Node.js med Express.

Database: I må *ikke* ændre i skemaet på det data warehouse, I har fået udleveret. I må gerne oprette views eller materialiserede views, hvis det hjælper jer.

Format: Alle svar fra API'et skal returneres som JSON.

Delopgave 1: Forbindelse til databasen

- ETL-databasen I skal bygge videre på, ligger i følgende repository:
<https://github.com/NervousPapaya/Specialisterne-Case---ETL-Pipeline>
- Opret jeres egen klon af repoet, så I kan arbejde uafhængigt. Følg derefter README'en for at sætte databasen op – den kan køres både lokalt og i Docker. Når I har databasen kørende og kan forespørge data, er I klar til at begynde på API'et.

Delopgave 2: Endpoints og filtrering

- I behøver ikke lave CRUD-operationer. Dataene vedligeholdes af ETL-pipelinen, og jeres API skal udelukkende læse fra databasen. Fokusér på vel designede GET-endpoints, fx:
 - Et endpoint der returnerer en liste over målestationer
 - Et endpoint der returnerer målinger for én station, med mulighed for at filtrere på tidsinterval og målingstype
 - Et endpoint der returnerer den seneste måling fra hver station
 - Et endpoint der gør det nemt at sammenligne data på tværs af stationer



- Lad brugeren styre hvad der returneres via query parameters, fx `?from=2024-01-01&to=2024-01-31` eller `?type=temperature`. Sørg for at returnere en fornuftig fejlbesked, hvis parametrene er ugyldige.

Delopgave 3: Dokumentation

- Sørg for at en anden kan bruge jeres API uden at spørge jer. Det kan løses på flere måder – mange frameworks genererer dokumentation automatisk (fx Swagger/OpenAPI i FastAPI), eller I kan skrive en udførlig README med eksempler på kald. Det vigtige er at det er tydeligt hvilke endpoints der findes, hvilke parametre de tager, og hvad de returnerer.

Valgfrie ekstraopgaver

- **Autentificering:** Implementér simpel API-nøgleautentificering, så ikke alle kan tilgå API'et uden videre
- **Containerisering:** Pak API'et i en Docker-container med en `docker-compose.yml`, der også starter databasen op
- **En simpel frontend:** Lav en simpel webside, der kalder API'et og viser data – fx en tabel eller en graf over temperaturudviklingen.
- **Rate limiting:** Begræns antallet af kald pr. minut pr. klient
- **Caching:** Implementér caching af hyppigt forespurgte data, så databasen ikke overbelastes
- **Test:** Skriv enhedstest og integrationstest for jeres endpoints

Pensum og Ressourcer

- [HTTP](#)



- [REST API](#)
-